Correction

Chaque correcteur prend en compte dans l'attribution de la note la qualité rédactionnelle des candidats : l'orthographe, syntaxe, grammaire, clarté de la langue et lisibilité du propos. Ainsi, toute copie dont la lecture serait jugée incompréhensible doit se voir attribuer une note inférieure à la moyenne. La situation particulière des candidats bénéficiant d'un aménagement ou adaptation doit naturellement être prise en compte

Correction

1	0.5	insertion dans une table	`INSERT INTO choregraphie` `VALUE ('Tout autour',2015,9);`

2	0.5	compter nbr lignes	`SELECT COUNT(nom)`
			`FROM choregraphie`
			`WHERE annee = 1983;`

3	0.5	comprendre une requête avec une jointure	Elle permet d'afficher le nom de toutes les chorégraphies avec le nom du chorégraphe associé.

4	1	écrire une requête avec jointures	`SELECT personne.nom`
			`FROM personne`
			`JOIN participe ON participe.idpersonne = personne.id`
			`JOIN spectacle ON participe.idspectacle = spectacle.id`
			`WHERE spectacle.titre = 'Tout autour';`

5	0.5	comprendre les contraintes d'une clé étrangère	Comme l'attribut `idspectacle` de la table `participe` est une clé étrangère faisant référence à l'attribut `id` de la table `spectacle`, il faut supprimer les enregistrements associés dans la table `participe`.

6	0.25	récupération d'infos dans un dico & compteur	`n = 0`
			`for clef in dmateriel:`
			`    n += dmateriel[clef]['nb']`

7	0.5	parcours d'un dico	`def tab_clefs_tab_volumes(dmatos):`
			`    tab_clefs = []`
			`    tab_volumes = []`
			`    for clef, dico in dmatos.items():`
			`        tab_clefs.append(clef)`
			`        tab_volumes.append(dico['volume'])`
			`    return tab_clefs, tab_volumes`

8	0.25	algo glouton	Algorithme glouton
9	0.25	algo glouton	Tri fusion ($n\log(n)$) ou tout autre algorithme de tri avec son coût associé.

10	1.25	mis en oeuvre de l'algo glouton	0.25 pour chaque trou
			`def remplissage(dmatos, vmax):`
			`    vol = 0`
			`    clefs_triees = clefs_triees_selon_volume(dmatos)`
			`    i = len(clefs_triees)-1`
			`    elements = []`
			`    while i >= 0 and vol < vmax:`
			`        clef = clefs_triees[i]`
			`        if dmatos[clef]['nb'] == 0:`

<table>
<tr><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td>

```
            i -= 1
        elif vol + dmatos[clef]['volume'] <= vmax:
            vol += dmatos[clef]['volume']
            elements.append(clef)
            dmatos[clef]['nb'] -= 1
        else:
            i -= 1
    return elements
```

</td></tr>
<tr><td>11</td><td>0.5</td><td>écrire un programme</td><td>

```
def nb_voitures(dmatos, vmax):
    n = 0
    while remplissage(dmatos, vmax) != []:
        n = n + 1
    return n
```

On acceptera tout algorithme renvoyant le bon résultat à + ou -1 près.
</td></tr>
</table>

Exercice 2 (6 points)

Correction

1	0.5	Compréhension fonctionnement perceptron	$somme = 1 \times 0.7 + 3 \times (-0.2) + 0.1 = 0.2$ ce qui donne la sortie à 1 0,25 pour la somme et 0,25 pour la sortie
2	0.5	POO : vocabulaire de la POO	`get_biais` (ou `set_biais` ou …) est une méthode et `biais` (ou `poids`) est un attribut 0,25 par réponse

3	0.75	POO : utilisation classe et méthodes	`>>> detecteur = DetecteurSpam(2)`
			`>>> detecteur.set_poids([0.7, -0.2])`
			`>>> detecteur.set_biais(0.1)`
			0,25 par ligne

4	0.25	Mise au point des programmes	Cette ligne de test permet de lever une erreur si le nombre de poids en paramètre est différent du nombre de poids du détecteur.

5	0.5	POO : écriture méthode	`def fonction_activation(self, valeur):`
			`    if valeur >= 0:`
			`        return 1`

			4 else: 5 return 0 ou toute autre solution correcte (exemple: `return valeur >= 0`) 0,25 pour la 1ère ligne et 0,25 pour le reste
6	1	POO : complétion méthode	1 def prediction(self, entrees): 2 somme = 0 3 for i in range(len(entrees)): 4 somme += entrees[i] * self.poids[i] 5 somme += self.biais 6 sortie = self.fonction_activation(somme) 7 return sortie 0,25 par trou
7	0.25	Détermination du coût d'exécution	Le coût d'exécution temporel de la fonction est linéaire ou en $O(n)$ (aucune justification attendue)
8	0.25	Listes Python	1 [0.2, 0.1] 2 0.8 0,25 pour les deux réponses correctes
9	1	POO : complétion d'une méthode	1 def entrainement(self, echantillon, cibles, taux=0.1): 2 nb_erreurs = 0 3 for i in range(len(echantillon)):

			0,25 par ligne complétée
10	1	Écriture d'un programme	

```
            entrees = echantillon[i]
            cible = cibles[i]
            # calcul de la prédiction
            predict = self.prediction(entrees)
            if predict != cible:
                # prédiction incorrecte
                nb_erreurs += 1
                erreur = cible - predict
                for j in range(len(self.poids)):
                    self.poids[j] += taux * erreur * entrees[j]
                self.biais += taux * erreur
        return nb_erreurs
```

```
detecteur = DetecteurSpam(2)
emails = [[0, 0], [0, 1], [1, 0], [1, 1]]
spams = [0, 0, 0, 1]
nb_erreurs = -1
nb_epoques = 0
while nb_erreurs != 0 and nb_epoques < 1000:
    nb_erreurs = detecteur.entrainement(emails, spams)
    nb_epoques += 1
if nb_erreurs != 0:
```

| | | | ```
    nb_epoques = -1
print(nb_epoques)
```<br><br>On valorise la présence de lignes correctes. On peut également tolérer quelques erreurs dans le code. |
```

Exercice 3 (8 points)

Correction

<table>
<tr><td></td><td></td><td></td><td></td></tr>
<tr><td>1</td><td>0.25</td><td>réseau</td><td>La partie fixe est <code>192.168.20.</code></td></tr>
<tr><td>2</td><td>0.5</td><td>binaire</td><td>Il y a 2 bits libres, donc 4 valeurs possibles. En enlevant les 2 adresses réservées, il est possible de mettre 2 machines.</td></tr>
<tr><td>3</td><td>0.75</td><td>protocole RIP</td><td>Table de routage de R2
<table><tr><th>Destination</th><th>Interface</th><th>Passerelle</th><th>Distance</th></tr><tr><td>192.168.20.0/24</td><td>172.24.0.2</td><td>172.24.0.1</td><td>1</td></tr><tr><td>172.24.0.0/30</td><td>172.24.0.2</td><td>–</td><td>0</td></tr><tr><td>172.24.1.0/30</td><td>172.24.1.2</td><td>–</td><td>0</td></tr><tr><td>192.168.40.0/24</td><td>172.24.1.2</td><td>172.24.1.1</td><td>1</td></tr></table></td></tr>
<tr><td>4</td><td>0.5</td><td>binaire</td><td>Le dernier octet est <code>11111100</code>, ce qui correspond à 252. Le masque est donc <code>(255, 255, 255, 252)</code>.</td></tr>
<tr><td>5</td><td>0.5</td><td>binaire</td><td><code>255</code> s’écrit <code>11111111</code> en binaire. Or <code>1 & 0 = 0</code> et <code>1 & 1 = 1</code>. Donc les bits du résultat sont exactement les mêmes que ceux de <code>A</code>.</td></tr>
<tr><td>6</td><td>0.75</td><td>binaire</td><td>Les deux premiers octets sont les mêmes et le dernier est libre. Il faut comparer au niveau du troisième octet.</td></tr>
</table>

<table>
<tr><td></td><td></td><td></td><td></td></tr>
<tr><td></td><td></td><td></td><td>
<pre>
   11111100 (252)   11111100 (252)
ET 10111101 (189)   ET 10111111 (191)
-----------   -----------
   10111100   10111100
</pre>
On obtient le même résultat, donc les deux adresses sont bien dans le même réseau.
</td></tr>
<tr><td>7</td><td>1.25</td><td>programmation</td><td>
<pre>
1 def meme_reseau(adresse1, adresse2, masque):
2   for i in range(4):
3   octet1 = adresse1[i] & masque[i]
4   octet2 = adresse2[i] & masque[i]
5   if octet1 != octet2:
6   return False
7   return True
</pre>
0,25 par ligne
</td></tr>
<tr><td>8</td><td>1</td><td>programmation</td><td>
<pre>
1   def diffuser(self):
2   while not self.file.est_vide():
3   paquet = self.file.defiler()
4   orig, donnees = paquet
5   for m in self.machines:
6   if m != orig:
7   self.machines[m].enfiler(paquet)
</pre>
0,25 par trou
</td></tr>
</table>

9	0.5	appliquer un protocole	`>>> routeur_m.table`
			`{ (172, 64, 0, 0): (1, None, 0),`
			`(172, 60, 0, 0): (0, (172, 58, 0, 2), 1),`
			`(192, 168, 47, 0): (0, (172, 58, 0, 2), 2),`
			`(172, 62, 0, 0): (1, (172, 64, 0, 2), 1),`
			`(192, 168, 72, 0): (1, (172, 64, 0, 2), 1)}`

10	0.5	programmation	`def m_a_j(self, passerelle, i_interf, vecteurs):`
			`a_ete_modif = False`
			`for adresse, d in vecteurs:`
			`if adresse not in self.table or d+1 < self.table[adresse][2]:`
			`self.table[adresse] = (i_interf, passerelle, d+1)`
			`a_ete_modif = True`
			`if a_ete_modif:`
			`self.envoyer_vecteurs(i_interf)`
			
			0,25 par ligne à compléter

11	0.5	programmation	`def vecteurs(self, i):`
			`res = []`
			`for adresse in self.table:`
			`i_interf, passerelle, dist = self.table[adresse]`
			`if i != i_interf:`
			`res.append((adresse, dist))`
			`return res`
			
			0,25 par ligne à compléter

12	0.5	programmation	1 `def envoyer_vecteurs(self, i_interf):` 2 `for i in range(len(self.interfaces)):` 3 `if i != i_interf:` 4 `adresse = self.interfaces[i].adresse` 5 `v = self.vecteurs(i)` 6 `paquet = (adresse, v)` 7 `self.interfaces[i].sortie.enfiler(paquet)` 0,25 pour les lignes 3 et 5, 0,25 pour la ligne 7
13	0.5	programmation	1 `for routeur in routeurs:` 2 `routeur.envoyer_vecteurs(-1)` 3 4 `continuer = True` 5 `while continuer:` 6 `continuer = False` 7 `for r in reseaux:` 8 `if not r.file.est_vide():` 9 `continuer = True` 10 `r.diffuser()` 11 `for routeur in routeurs:` 12 `routeur.traitement()`